Technical Documentation

Medical Delivery Drone for Critical Supply Transport

GitHub / engineering repository documentation

Implemented baseline	Quadrotor prototype described in the final report
Primary architecture	Pixhawk 6C + Raspberry Pi 5 companion-computing system
Power architecture	Split 4S propulsion rail + 3S companion rail with 5 V / 15 A UBEC
Documentation purpose	Describe build, wiring logic, software structure, and image-based evidence

Demo Vido Link: https://youtube.com/shorts/6_gysADudQQ?si=MeAfai4QAqJnp9QC

1. System overview

This document explains how the implemented medical-delivery drone prototype is built, how the main subsystems are connected, and how the companion-computer software is organised. It is written as technical repository documentation rather than as a flight-operation approval document.

The implemented baseline is a quadrotor centred on a Pixhawk 6C flight controller and a Raspberry Pi 5 companion computer. The system is divided into a propulsion layer, a flight-control layer, and a companion-computing layer.

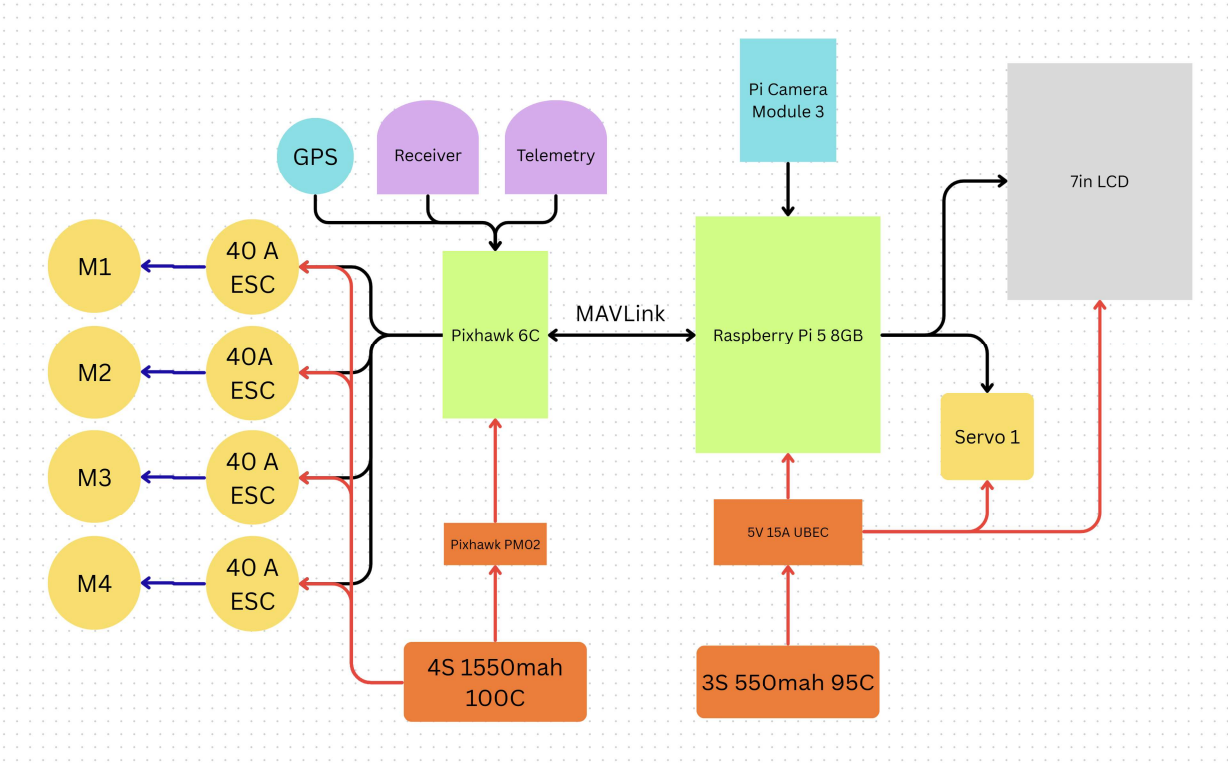

Figure 1. Implemented hardware block diagram used in the final report.

2. Main hardware used

Subsystem	Main hardware	Role
Flight control	Pixhawk 6C, GPS/GNSS, ELRS receiver, telemetry radio	Stabilisation, state estimation, radio-control compatibility
Propulsion	4 × 2807 1300 KV motors, 4 × XRotor 40 A ESCs, Gemfan 9045 props	Lift generation and thrust control
Companion computing	Raspberry Pi 5, Pi Camera Module 3, 7-inch LCD, payload servo	GUI, mission logic, recording, payload handling
Power	4S 1550 mAh 100C, PM02, 3S 550 mAh 95C, 5 V / 15 A UBEC	Separate propulsion and companion power domains

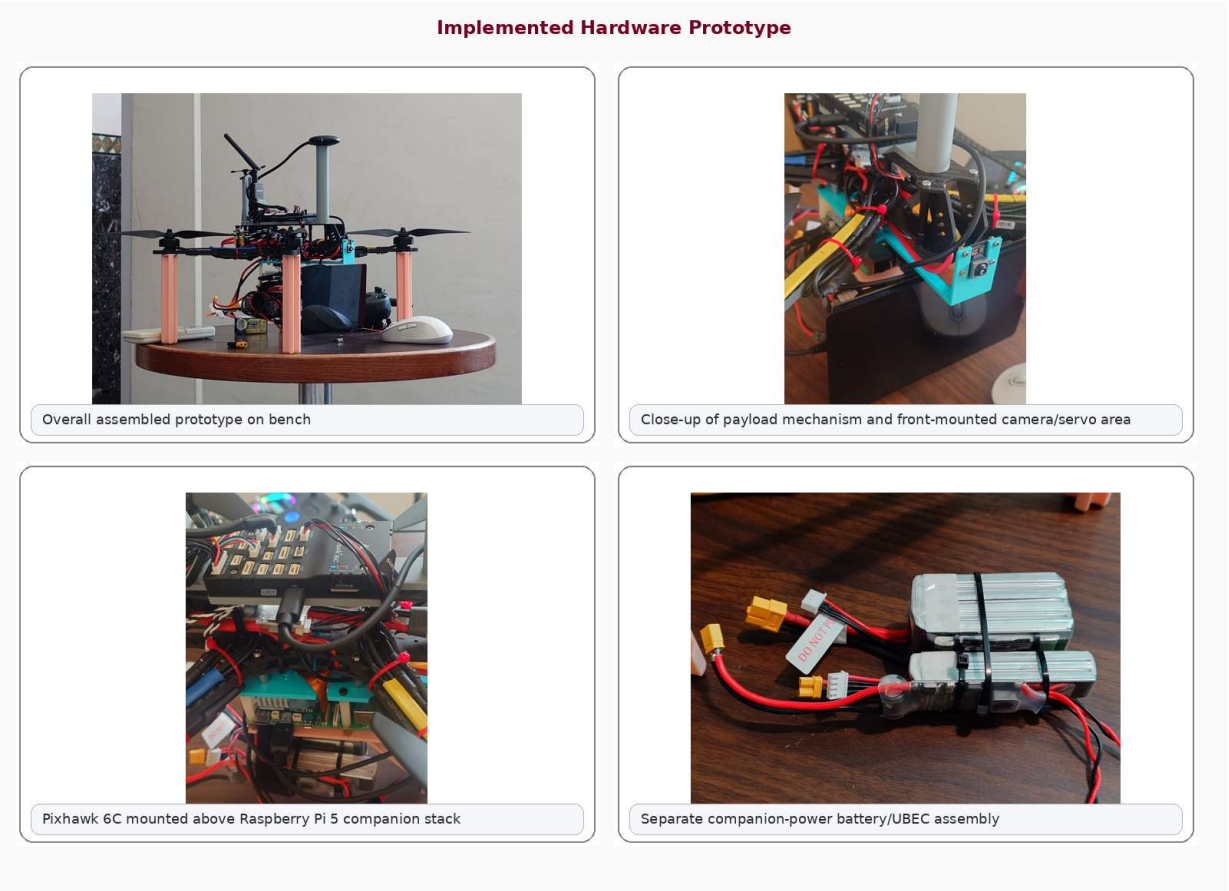

Figure 2. Hardware evidence collage showing the assembled prototype, payload/camera area, Pixhawk/Raspberry Pi stack, and separate companion-power battery/UBEC assembly.

3. Connection and wiring logic

3.1 Propulsion-side connections

- The 4S propulsion battery feeds the propulsion rail.
- The propulsion rail powers all four ESCs.
- Each ESC drives one motor.
- The PM02 draws from the propulsion side and feeds the Pixhawk power path while exposing voltage/current sensing.

3.2 Flight-controller connections

- GPS / GNSS module → Pixhawk GPS port
- ELRS receiver → Pixhawk RC input path
- Telemetry radio → Pixhawk telemetry port
- Pixhawk USB → Raspberry Pi USB for MAVLink communication

3.3 Companion-computing connections

- 3S companion battery → 5 V / 15 A UBEC
- UBEC 5 V output → Raspberry Pi 5
- UBEC 5 V output → 7-inch LCD
- UBEC 5 V output → payload servo
- Pi Camera Module 3 → Raspberry Pi camera interface

3.4 Why split power is used

The propulsion battery and the companion-computer battery are intentionally separated. This reduces the chance that propulsion-side voltage sag or ESC noise propagates into the Raspberry Pi rail. In practice, this lowers the risk of Pi undervoltage events or resets during throttle transients and makes the system easier to troubleshoot.

4. Physical build arrangement

- Top / airframe level: quad arms, motors, propellers, GPS mast.
- Middle avionics level: Pixhawk 6C, receiver, telemetry, PM02-related wiring.
- Lower companion-computing level: Raspberry Pi 5, LCD path, camera path, payload-control path.
- Battery arrangement: separate propulsion battery and separate companion battery.

The layered arrangement improves serviceability because propulsion, flight-control, and Pi-side electronics can be inspected as semi-separate subsystems.

5. Software architecture

The implemented baseline described in the final report is the v50 Light control application. The Raspberry Pi is treated as a companion computer, not as the primary flight controller. The Pi reads telemetry over MAVLink, runs the GUI and mission logic, and supervises payload state and return behaviour, while Pixhawk remains responsible for low-level flight stability.

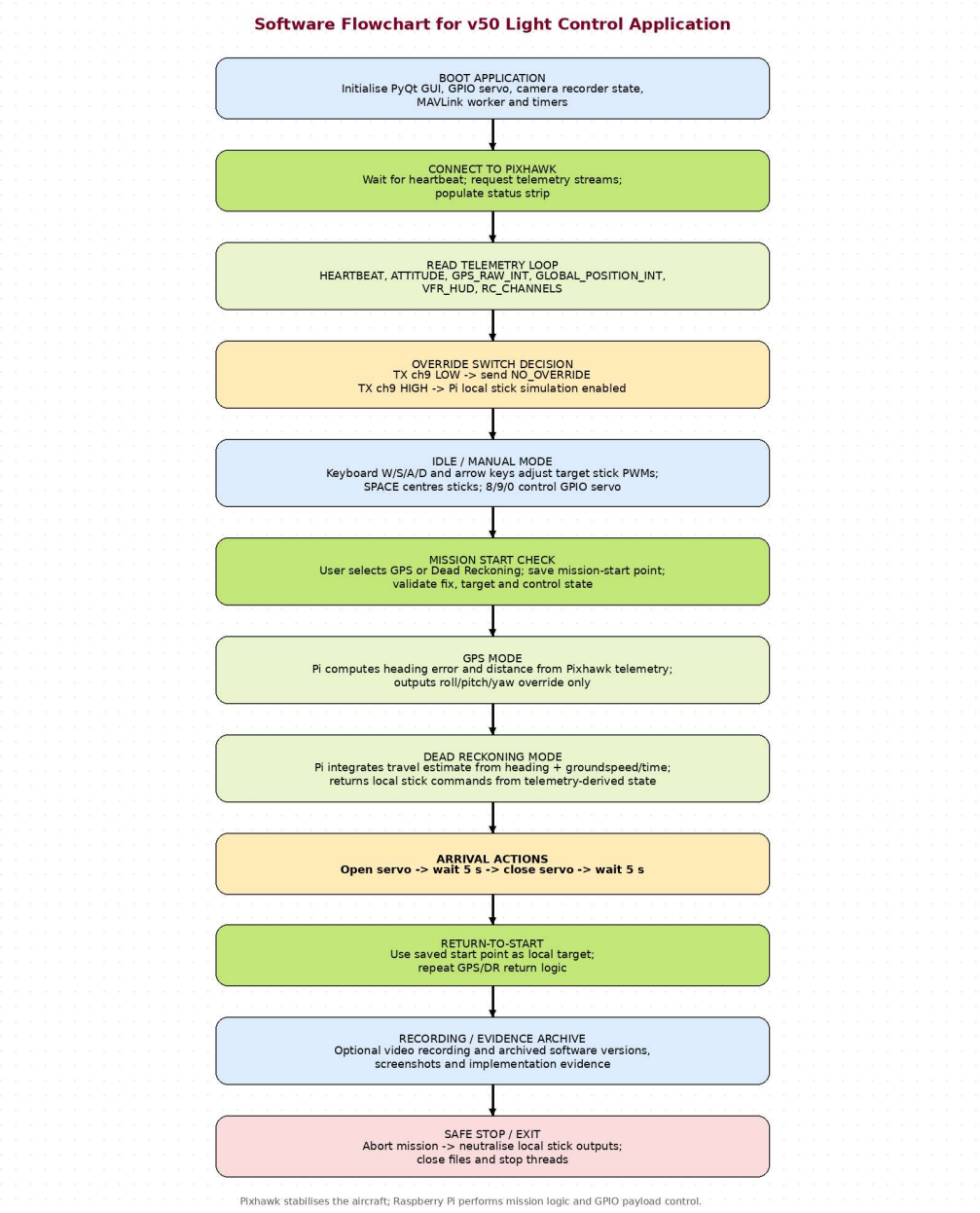

Figure 3. Software flowchart for the v50 Light control application.

5.1 Main software modules

- MAVLink communication worker
- PyQt dashboard and mission tabs
- Manual local-stick / keyboard logic
- GPS mission page
- Dead-reckoning page
- Payload release state machine
- Return-to-start logic

- Recording and evidence archive support

5.2 Mission and payload logic

At mission target arrival, the payload sequence is defined explicitly: the servo opens the release mechanism, remains open for a timed dwell, closes again, and the software then begins return-to-start logic. This gives the project a clear operational narrative instead of leaving payload handling as an undefined concept.

5.3 Why the software scope was simplified

Continuous subject detection and obstacle detection were deliberately removed from the main mission path because the intended use case is an outdoor, above-ground delivery profile. This simplification reduced software complexity and focused the prototype on telemetry reliability, mission-state clarity, payload sequencing, and navigation logic.

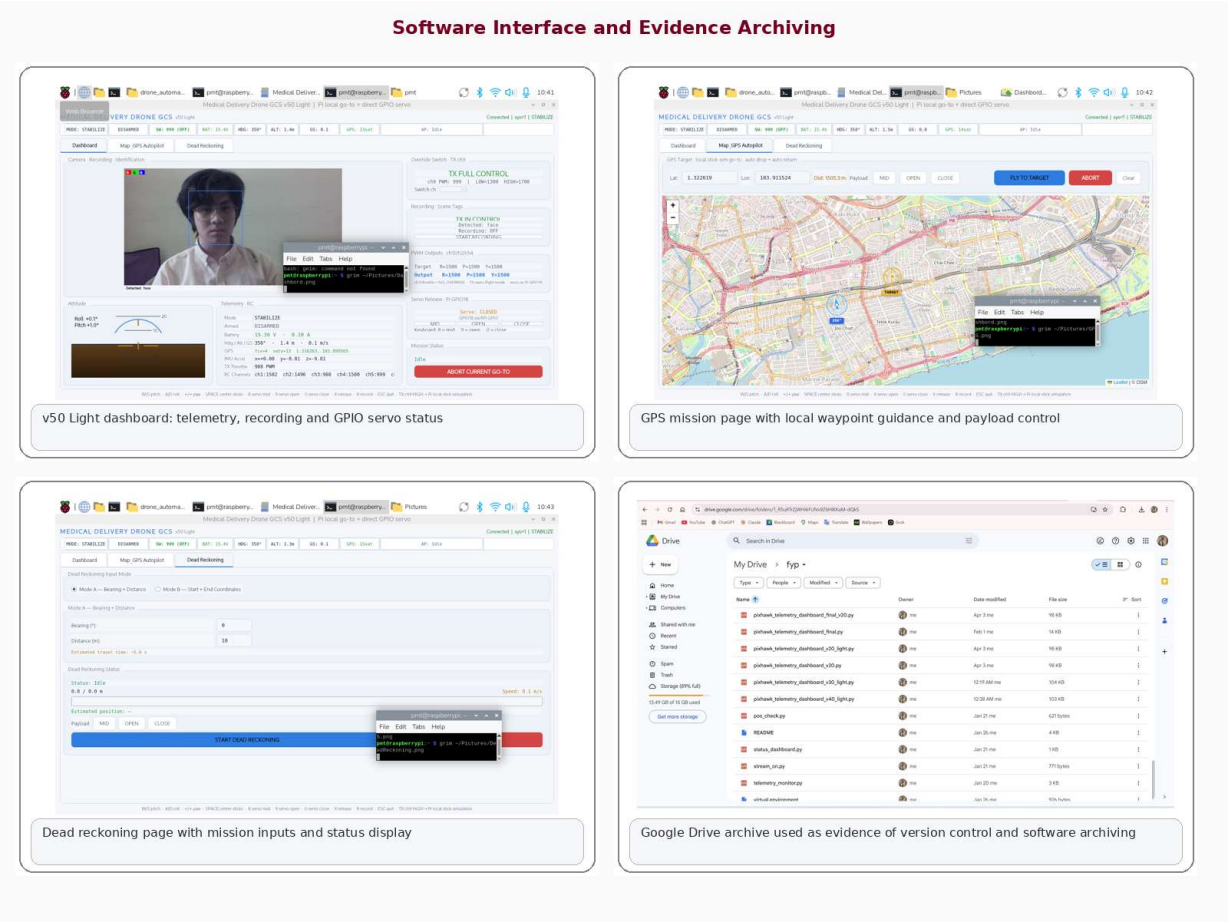

Figure 4. Delivered interface views and software archive evidence used in the report.

6. Repository software variants

Version	Takeoff	Mission start	Landing
Version 1	Manual	Started manually in GUI after takeoff	Manual
Version 2	Automatic	Started in GUI; full autonomous mission flow	Automatic
Version 3	Manual	Begins automatically after reaching 4 m altitude	Manual

7. Setup note for repository users

Before running the software, the Pixhawk serial device path in the Python file should be updated to match the connected flight controller. For example:

```
PORT = "/dev/serial/by-id/usb-Holybro_Pixhawk6C_xxxxxxxxxxxxxxxxx-if00"
```

To find the correct serial ID on the Raspberry Pi:

```
ls /dev/serial/by-id/
```

8. Safety note

This documentation describes an academic prototype. Any real testing should be performed with appropriate safety controls, regulatory awareness, and bench-testing discipline. Software bring-up and wiring validation should always be carried out with propellers removed.